Homework 10

Steven Jia

November 1, 2026

Question 1: Bias and variance

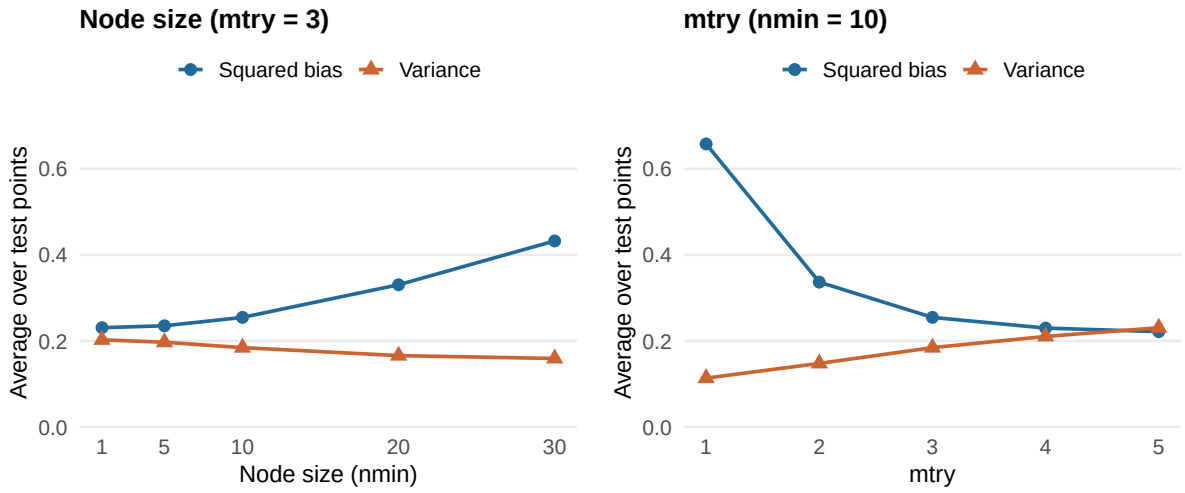
(a) Simulation and results

I generated five jointly normal predictors with means 0, variances 1, and pairwise correlations 0.3, and $Y = X_1 + X_3 + X_5 + \varepsilon$ with independent $\varepsilon \sim N(0,1)$. Using seed **43210**, I fixed 200 test points and generated 100 training sets of 200 observations. All settings shared each training set. In RLT 6.1.1, I used 500 trees per forest, 200 bootstrap draws per tree, and reinforcement off, leaving other fitting settings at their defaults. Forest seeds were $43210 + 1000b + s$, for repetition b and setting index s.

At test point x_j, I computed

$$\bar{f}_j = \frac{1}{100}\sum_{b=1}^{100} \hat{f}_b(x_j), \qquad \widehat{\mathrm{Bias}^2}_j = (\bar{f}_j - f(x_j))^2, \qquad \widehat{\mathrm{Var}}_j = \frac{1}{99}\sum_{b=1}^{100}(\hat{f}_b(x_j) - \bar{f}_j)^2.$$

The figures average these estimates over the 200 test points. Bias is measured against $f(x) = x_1 + x_3 + x_5$, without adding test-response noise.



(b) Interpretation

Node size. With mtry fixed at 3, increasing node size from 1 to 30 raised squared bias from 0.231 to 0.433, while variance fell from 0.203 to 0.159. A node stops splitting at nmin observations or fewer. A value of 30 stops trees earlier and makes the fit less flexible. I think this underfits here: the increase in bias outweighs the reduction in variance.

mtry. Increasing mtry from 1 to 5 reduced squared bias from 0.658 to 0.222, but increased variance from 0.114 to 0.231. More candidates make useful predictors available at more splits, but can make trees more similar and reduce the benefit of averaging. They also require more split searches. I found similar balances at mtry 3 and 4, with the smallest empirical error against $f(x)$ at 3 in this comparison.

Sampling 100 observations per tree. I would expect smaller node sizes to help: the same threshold is now a larger fraction of the sample and can stop splitting too early. A smaller threshold can reduce this extra bias. However, fewer observations also increase instability, so the best node size is not guaranteed to decrease. I did not run an additional simulation for this part.

Question 2: Random forests for MNIST

Data and preprocessing

I kept digits 1, 6, and 7 from the first 1,000 observations for training, and the same digits from the remaining 1,000 for testing. This gave **327 training observations and 317 test observations**.

Digit	Training	Test
1	116	104
6	94	106
7	117	107

I fitted PCA to the 784 training pixels, centering but not scaling them. The first 20 principal components explained 77.13% of the training pixel variance. I projected test images using the same training means and loadings. All forests used 1,000 trees, reinforcement off, and 327 bootstrap draws per tree. I used seed **43211** for each forest and checked that the bootstrap samples matched across settings. The OOB rates below evaluate the forests after this training-set PCA.

(a) Default forest

The default settings in this RLT version are **mtry = 10 and node size = 5**. The OOB misclassification rate was **14/327 = 4.28%**.

(b) Tuning with OOB error

I compared mtry values 2, 4, 8, 10, 12, and 20, and node sizes 1, 5, 10, and 20. I selected the lowest OOB error. My rule for a tie was to choose smaller mtry, then larger node size. The table shows all 24 OOB error rates. Each rate uses the same 327 training observations.

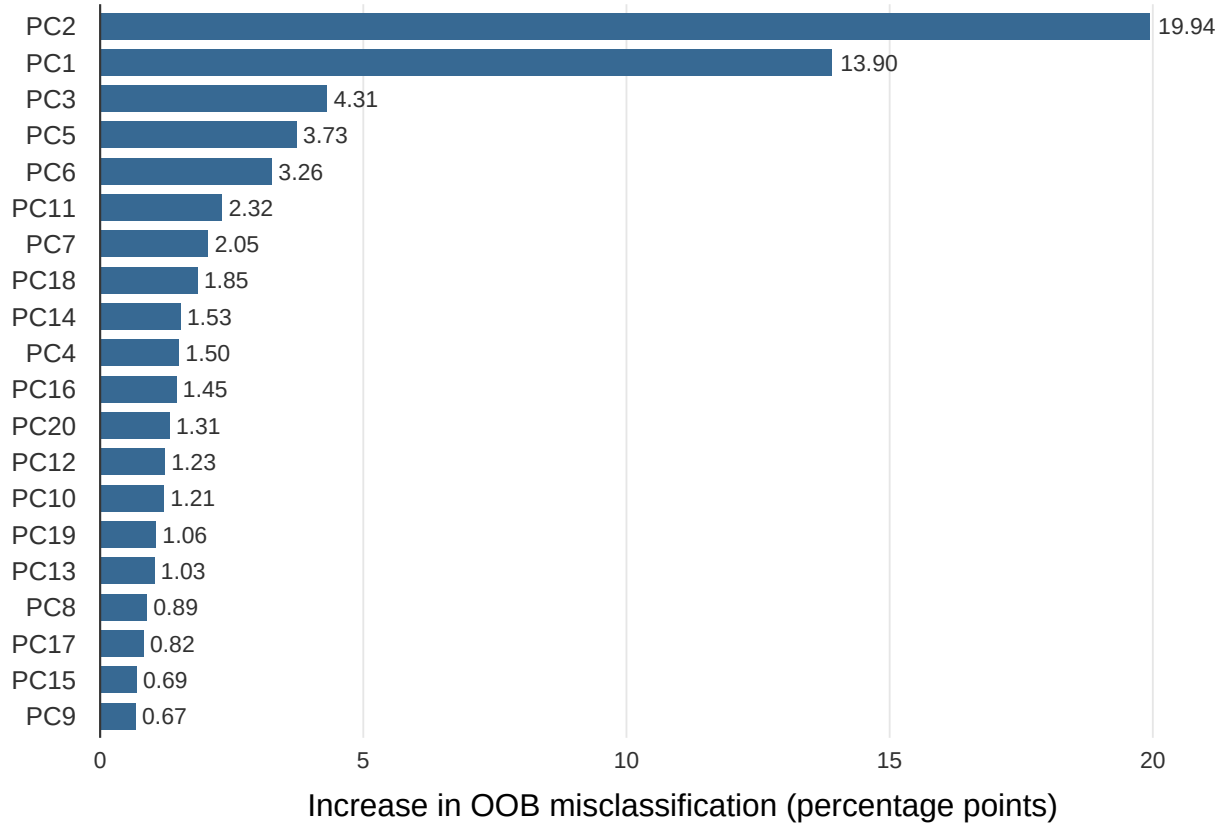
Table 1: OOB misclassification rates; * marks the selected setting.

mtry	Node size 1	Node size 5	Node size 10	Node size 20
2	2.75% *	3.06%	3.06%	3.36%
4	3.67%	3.67%	3.67%	3.67%
8	3.98%	3.98%	3.98%	3.98%
10	4.28%	4.28%	4.28%	4.28%
12	4.59%	4.59%	4.59%	4.59%
20	4.89%	4.89%	4.89%	5.20%

I selected **mtry = 2, node size = 1**. This was the unique lowest OOB error: **9/327 = 2.75%**. It was 1.53 percentage points below the default forest, or five fewer OOB errors. In this grid, smaller mtry worked better. Changing node size made less difference to the classification errors. Some settings had the same error count even though their predicted probabilities differed.

(b) OOB permutation importance

For each tree, I permuted one principal component among its OOB observations and measured the increase in misclassification. The figure averages this increase over the 1,000 trees. Values are in **percentage points**.



I found that **PC2 and PC1** were the most important, with increases of 19.94 and 13.90 percentage points. PC3 was next. The first two components were especially useful for separating these digits. The ranking was not exactly the PCA order: PCA explains pixel variance, while this importance measures predictive usefulness. These values describe this fitted model and do not show causal effects or add up to the forest's total error.

(c) Final test assessment

After saving the model choice, I evaluated the selected forest on the test set. It misclassified **6 out of 317** observations, giving a test error of **1.89%**. Rows below are actual digits; columns are predicted digits.

Actual digit	Predicted 1	Predicted 6	Predicted 7
1	102	0	2
6	0	106	0
7	0	4	103

Two 1s were predicted as 7, and four 7s were predicted as 6. All 106 actual 6s were classified correctly. The test error was lower than the OOB error, but these are different samples. I would not expect exactly the same result on another test set.

Question 3: Planning a random-forest analysis skill

(a) How I would write the skill

I do not think the skill needs to be very long. I would first tell the AI what to do and what kind of report I want. If the AI already knows how to use random forests, I probably do not need to explain the code and basic steps in much detail. But I would still remind it about a few things, like not using test results to choose the parameters. If the AI does not know how to do the analysis well, I would write more steps for it to follow. I care more about whether I can understand the results at the end. So I would ask for simple tables and figures and explanations that are not too long. It should tell me which model was chosen and why.

(b) Ten instructions, from basic to personal

1. Look at the variables in the data and decide what to predict.
2. Check for missing values or anything that looks wrong in the data.
3. Split the data into training and test sets. Keep the test set for the end.
4. Set a fixed random seed so the analysis can be run again later.
5. Run a random forest with the default settings first and see how it does.
6. Try a few values of mtry and node size, and choose using OOB error. Do not change the parameters based on test results.
7. Check the chosen model on the test set, report the error, and briefly say how it did.
8. Put the results for different parameter settings in a simple table. I do not want to read a long paragraph of numbers.
9. Include one or two useful figures, such as a plot of which variables are important. There is no need to plot every result.
10. Keep the explanation simple. Tell the user what was chosen and why, and do not make it too long.